Cybersecurity + Process Injection + Malware Development +

DLLParty — Abusing thread pool internals for shellcodeless DLL injection

11 min read · 5 days ago

E epersic Follow

Listen Share

Motivation

Recently I was experimenting with vibecoding custom beacons, injecting beacon code into common open-source DLLs and compiling the malicious code together with legitimate DLL code as a way to avoid having to deal with unbacked memory and “dirty” callstacks. Having your beacon compiled as part of a DLL rather than as shellcode has its pros and cons, one of the major cons being you lose the process injection options, as most modern process injection techniques rely on your code being in a shellcode format.

Previously when thread pools were used to perform a DLL injection it was always through an assembly trampoline or shellcode, like in this research here from Paranoid Ninja.

In this blog I am going to try and answer the following question: Is it possible to coerce Windows internals to load the DLL for you without using any shellcode?

PS: This technique builds heavily on PoolParty technique, with one key difference — We won’t use any shellcode at all! No code execution from unbacked memory and in certain cases even no calls to VirtualAllocEx.

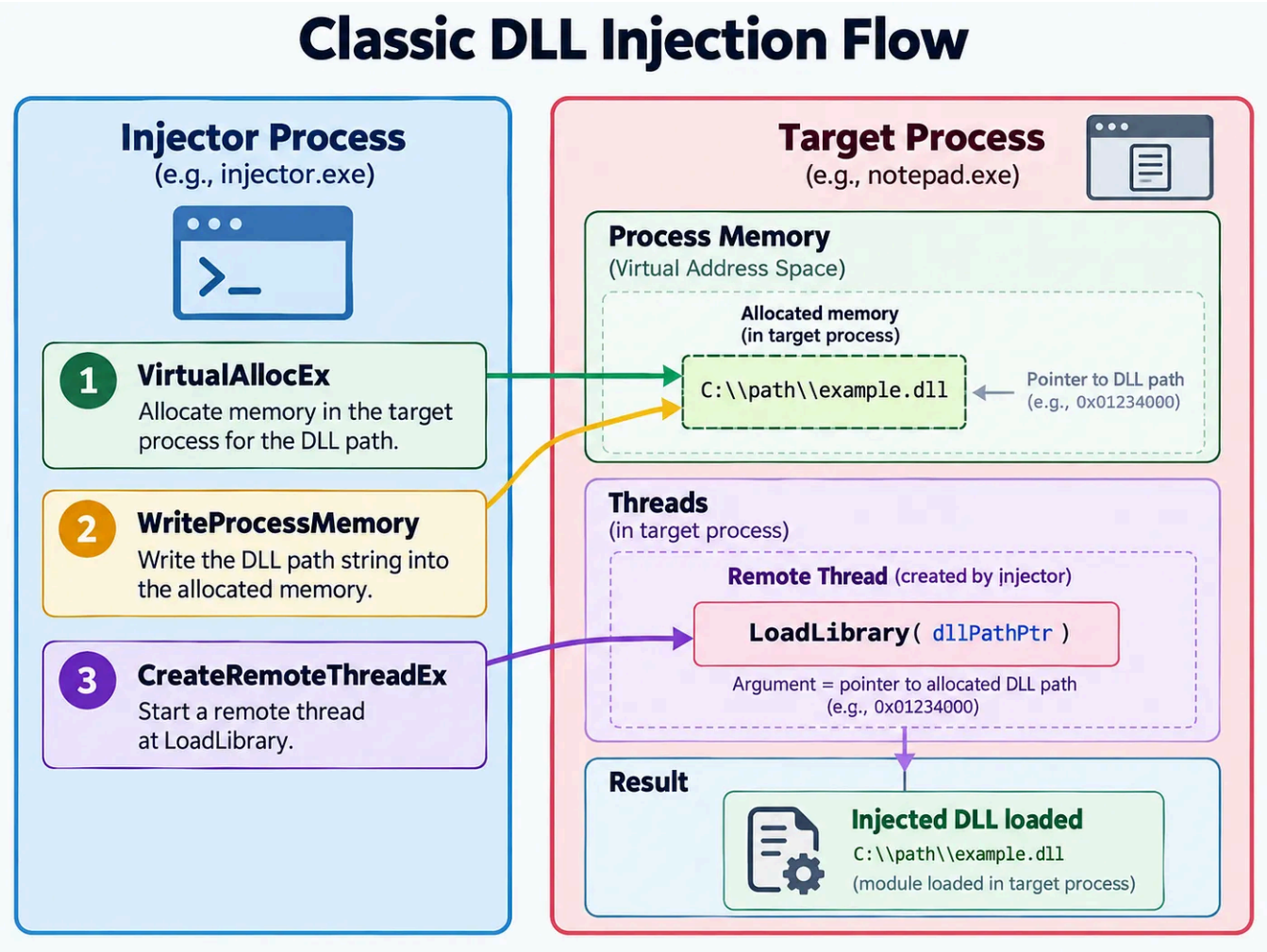
Classic DLL injection and why it’s bad

Classic DLL injection has the following flow:

1. Allocate the memory in the process you want to inject the DLL to with VirtualAllocEx, this place will be used to store the path of the DLL we want to inject.

2. Use WriteProcessMemory to write the DLLpath into the allocated memory from the first step.

3. Call CreateRemoteThreadEx with the following arguments: Pointer to LoadLibrary function and a pointer to dllPath string that we stored earlier. This will create a thread in another process that will execute LoadLibrary function loading the DLL from our path into the targeted process.



This combination of APIs is strongly associated with injection with the most critical one being CreateRemoteThreadEx, very few legitimate processes have a legitimate need to create a thread in another process, mostly debuggers and other development tools. Windows has kernel callbacks that trigger when a thread is created, seeing that the creator of the thread isn't the same process as the one the thread is being created in is a major red flag. Beyond classic Windows Kernel callbacks ETW TI allows for monitoring both VirtualAllocEx and

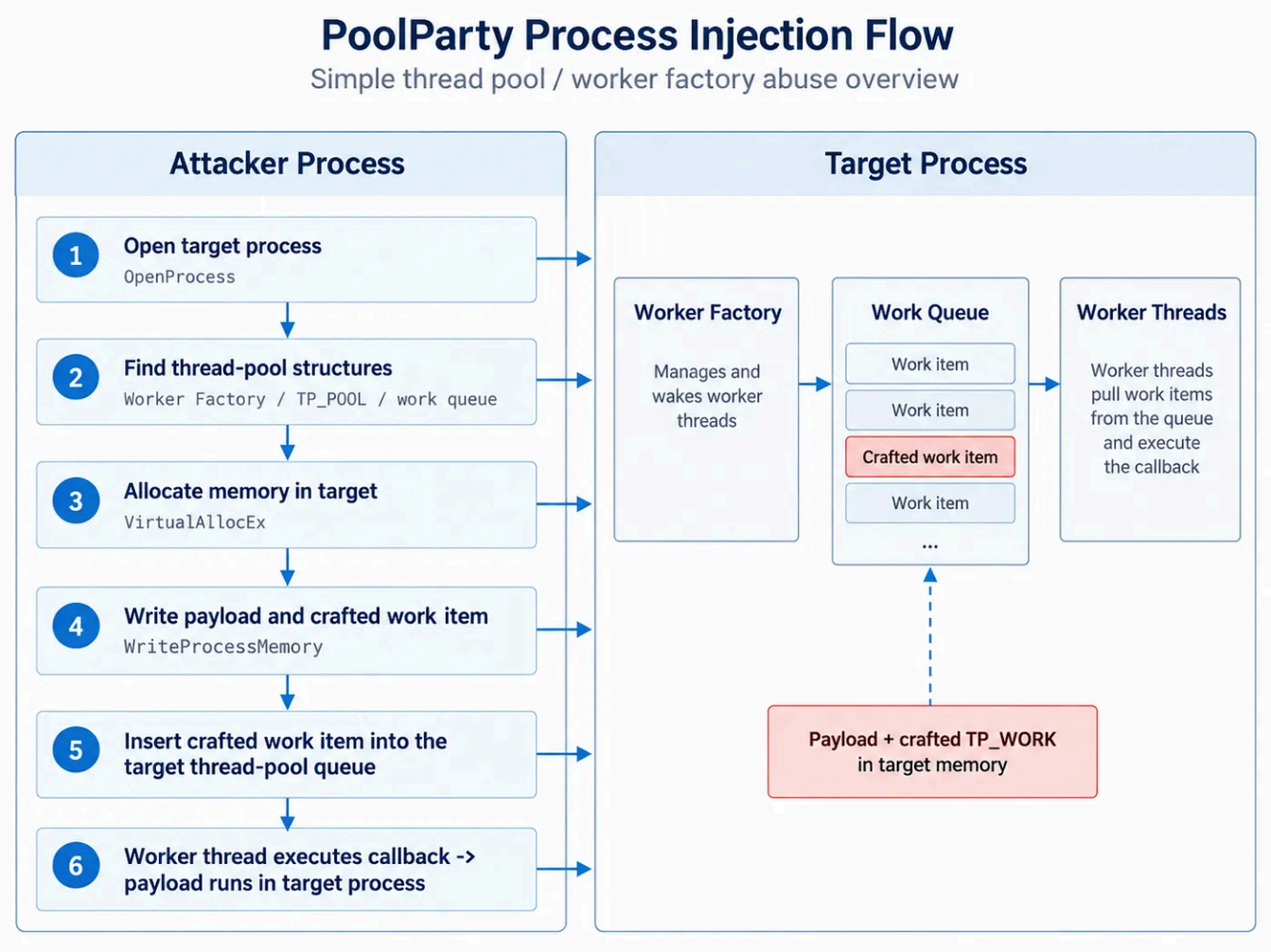
WriteProcessMemory calls. However, detecting only VirtualAllocEx or only WriteProcessMemory calls very often isn't a big enough indicator to raise the detection by itself.

Windows thread pools and Poolparty

I won't go into detail on how Windows thread pools or Poolparty technique work. I suggest reading the original Poolparty research for that, there is also this website that does a pretty good job of explaining it, even though there are some mistakes in pseudo code examples.

Brief overview: Windows thread pools are a feature where Windows keeps alive so-called "Worker threads", you can delegate any tasks you want to be executed asynchronously to them. The tasks are delegated in the form of callback functions. You give the thread pool a callback function you want to be executed, it puts it onto a queue and a worker thread that doesn't have a job currently fetches it and executes your callback function. This is done because the creation of new threads is expensive, this way the cost of smaller parallel tasks executed in separate threads is way smaller.

Poolparty technique abuses this feature by manually putting work onto another processe's thread pool queue (the queue is called IOCP), with the callback pointing to the shellcode that was previously written into the target process memory, rather than pointing to a legitimate callback function. That way the worker thread that picks up that callback will execute the shellcode.



DLLParty

When using thread pools legitimately, callback function needs to have one of the predefined signatures:

```
MyWaitCallback(
    PTP_CALLBACK_INSTANCE Instance,
    PVOID                 Parameter,
    PTP_WAIT              Wait,
    TP_WAIT_RESULT        WaitResult
    )
```

Thread pool wait callback signature

```
MyWorkCallback(
    PTP_CALLBACK_INSTANCE Instance,
    PVOID                 Parameter,
    PTP_WORK              Work
    )
```

Thread pool work callback signature

```
MyTimerCallback(
    PTP_CALLBACK_INSTANCE Instance,
    PVOID                 Parameter,
    PTP_TIMER             Timer
    )
```

Thread pool timer callback signature

These are just a few examples, there are more of them. The thing you may notice they all have in common is the first parameter: a pointer to TP_CALLBACK_INSTANCE structure. Let's take a look at LoadLibraryA definition.

```
HMODULE LoadLibraryA(
  [in] LPCSTR lpLibFileName
);
```

LoadLibraryA signature

The function takes only 1 argument, a pointer to the string. A pointer is simply an address in memory, whether it points to TP_CALLBACK_INSTANCE or a string is up to the function receiving the pointer, in this case LoadLibraryA and the callback function, to interpret it.

The idea is the following: If we can manipulate the TP_CALLBACK_INSTANCE object in a stable way and change it to "C:\path\to\dll.dll", path of the DLL we want to inject, it is possible to set the callback function directly to LoadLibraryA from

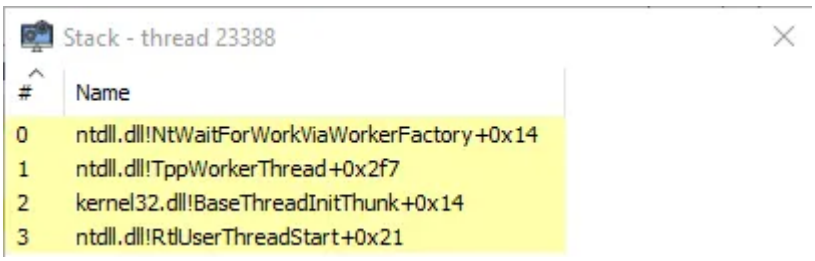
Open in app ↗ Sign up Sign in

Medium Search

The problem is TP_CALLBACK_INSTANCE is not user-controlled, it is internally built by Windows itself.

TP_CALLBACK_INSTANCE

Normal and expected worker thread call stack looks something like this:



Worker thread stack

TppWorkerThread function is the “main” function of every worker thread whose oversimplified code is this:

```
void TppWorkerThread(TP_POOL* pool)
{
    TP_CALLBACK_INSTANCE instance = {};

    RegisterWorkerWithPool(pool, &worker);

    while (true)
    {
        WaitForWork();

        TASK* task = GetNextTask(pool);
        if (!task)
            break;

        PopulateCallbackInstance(&instance, task);

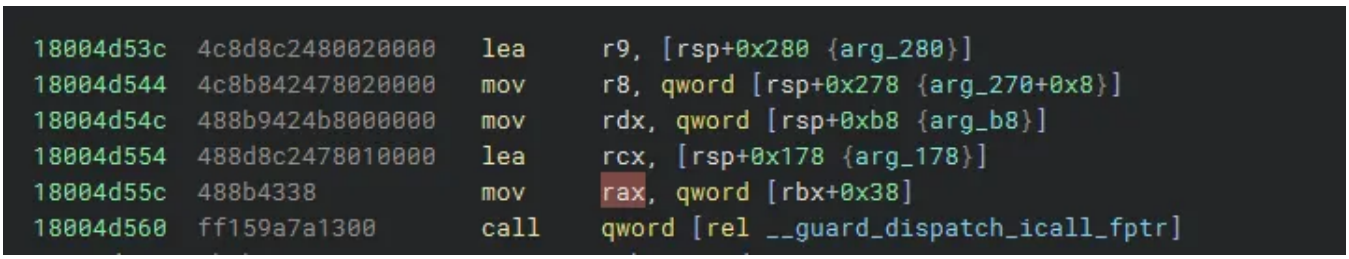
        task->Callback(&instance, task->Context);

        CleanupAndResetInstance(&instance);
    }

    UnregisterWorkerFromPool(pool);
}
```

This is highly simplified but it gives us a good intuitive picture of how the function works and what its tasks are. It has a while loop where it waits for tasks.

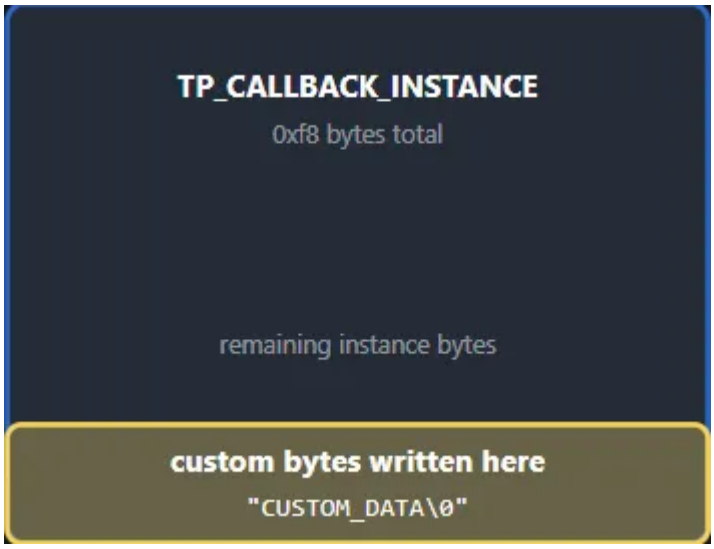
We can start from here and work back to determine exactly how and where pointer to TP_CALLBACK_INSTANCE is crafted.



Callback function invocation

This snippet of code shows where the callback function is being invoked, from here we can see that the pointer is crafted right before the invocation using x64 lea

instruction, since the first parameter (rcx on x64 calling convention), is located at rsp+0x178. This tells us two things: TP_CALLBACK_INSTANCE object is located on the stack (read/write memory by default) and object location remains stable relative to rsp register before the callback is executed in the TppWorkerThread function. This is good news because this means that we can locate the object while the worker thread is waiting to pick up some work. We can abuse this by first injecting our own bytes at rsp+0x178 and then adding work to the queue effectively controlling what the pointer points to.



Stack schema with injected bytes

The only thing that remains to be seen is how the TP_CALLBACK_INSTANCE object is actually populated, if the whole object is populated before the work is picked up our injected bytes will be overwritten. Since we know that TP_CALLBACK_INSTANCE object begins at rsp+0x178 we can simply look at all instructions writing to memory locations after rsp+0x178 to see how the object is populated.

The TP_CALLBACK_INSTANCE is populated in many steps, but the one we really only care about is the one that is the closest to rsp+0x178, because that will tell us if we have any space to work with regarding bytes that remain untouched during the population.

And as it turns out, most of the time we do!

After looking through the function the first address that seems to be populated inside of TP_CALLBACK_INSTANCE object is rsp+0x1d0. This means that we have

0x1d0–0x178(88) bytes of memory that remain untouched before the worker thread calls the callback function!

```
18004d67f  48898c24d0010000   mov     qword [rsp+0x1d0 {var_128_2}], rcx
18004d687  48899424d8010000   mov     qword [rsp+0x1d8 {var_120_2}], rdx
```

Populating of TP_CALLBACK_INSTANCE

The prefix is reserved for conditional internal callback bookkeeping. On the ordinary paths tested here, those fields were unused and remained zero.

Locating TP_CALLBACK_INSTANCE object

Now that we know that we can safely inject our own bytes to the beginning of the TP_CALLBACK_INSTANCE object and Windows will internally craft a pointer pointing to them, effectively passing a string to the function (DLL path in DLLinjection case) we need to be able to reliably locate the TP_CALLBACK_INSTANCE object.

Get epersic's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

The first approach would be to use the GetThreadContext function to read the RSP register directly. This approach has several downsides. Since we cannot tell which worker thread from the thread pool will pick up our work, we need to enumerate all the threads in the process and open handles to all of them and overwrite the bytes at rsp+0x178 on all threads stacks. Although unsuspended context reads frequently produced usable values during testing, Microsoft does not guarantee a valid context for a running thread. This approach is therefore inherently racy and was replaced by direct worker-list traversal.

Second approach doesn't require enumerating or opening handles to any of the threads. Lets take a look at what else is there on the stack inside of TppWorkerThread function frame.

At the beginning of the function 0x168 bytes are set to 0, starting from rsp+0x140.

```
18004d1c5  33d2              xor     edx, edx  {0x0}
18004d1c7  41b868010000      mov     r8d, 0x168
18004d1cd  488d8c2440010000  lea     rcx, [rsp+0x140 {var_1b8}]
18004d1d5  e8e6740500        call    memset
```

Initial stack initialization

You may notice that this is 0x38 bytes before the beginning of TP_CALLBACK_INSTANCE object (0x178–0x140). Those bytes are populated with metadata before the main while loop.

TppWorkerThread calls TppPoolAddWorker function and passes it pointer to rsp+0x140.

```
18004d2bc  488d942440010000  lea     rdx, [rsp+0x140 {var_1b8}]
18004d2c4  488bce            mov     rcx, rsi
18004d2c7  e8e4180000        call    TppPoolAddWorker
```

TppPoolAddWorker call

TppPoolAddWorker then inserts the pool worker-list head.

```
18004ebd7  44894228          mov     dword [rdx+0x28], r8d
18004ebdb  48894a30          mov     qword [rdx+0x30], rcx
```

Moves the pointer the the rsp+0x140 to rdi and rcx into rbx.

```
18004ebc4  488bfa            mov     rdi, rdx
18004ebc7  488bd9            mov     rbx, rcx
```

Loading of initial values

Loads that pointer into rax and offset 0x60 of rbx(previously rcx) back to rcx:

```
18004ebe9  488d4710          lea     rax, [rdi+0x10]
18004ebed  488d4b60          lea     rcx, [rbx+0x60]
```

And moves pointers that it just crafted into memory locations it just calculated:

```
18004ec01  488908            mov     qword [rax], rcx
18004ec04  48895008          mov     qword [rax+0x8], rdx
18004ec08  488902            mov     qword [rdx], rax
18004ec0b  48894108          mov     qword [rcx+0x8], rax
18004ec0f  488d4b48          lea     rcx, [rbx+0x48]
```

What this sequence actually just did was create or update a linked list.

The thing I didn't mention is what was in the rsi register from the first snippet "TppPoolAddWorker call" in this sequence. In RSI register was an address of TP_POOL object.

During classic poolparty we fetch TP_POOL object like this:

```
WORKER_FACTORY_BASIC_INFORMATION wfbi;
NtQueryInformationWorkerFactory_t NtQueryInformationWorkerFactory =
  (NtQueryInformationWorkerFactory_t)GetProcAddress(
    GetModuleHandleA("ntdll"),"NtQueryInformationWorkerFactory"
    );

NtQueryInformationWorkerFactory(
  wf_dup_handle,WorkerFactoryBasicInformation,&wfbi,sizeof(wfbi),NULL);

target_pool_ptr = wfbi.StartParameter;
target_worker_factory = wf_dup_handle;

FULL_TP_POOL_PREFIX pool = {0};
SIZE_T bytes_read =0;
ReadProcessMemory(
    proc_handle,target_pool_ptr,&pool,sizeof(pool),&bytes_read));
```

TP_POOL structure definition (not 100% correct but good enough):

```
typedef struct _FULL_TP_POOL_PREFIX {
    TPP_REFCOUNT Refcount;              // 0x00
    LONG Padding;                       // 0x04
    TPP_POOL_QUEUE_STATE QueueState;    // 0x08
    TPP_QUEUE *TaskQueue[3];            // 0x10
    TPP_NUMA_NODE *NumaNode;            // 0x28
    GROUP_AFFINITY *ProximityInfo;      // 0x30
    PVOID WorkerFactory;                // 0x38
    PVOID CompletionPort;               // 0x40
    SRWLOCK WorkerListLock;              // 0x48
    BYTE Unknown50[0x10];               // 0x50
```

```
    LIST_ENTRY WorkerListHead;          // 0x60
} FULL_TP_POOL_PREFIX;
```

So lets go through the assembly one more time:

```
mov rdx [rsp+0x140] //pointer to the structure before TP_CALLBACK_INSTANCE
                    //is passed to rdx
mov rcx,rsi //pointer to TP_POOL is moved to rcx(calling convention)
mov [rdx+0x30] rcx //stores pointer to TP_POOL to rsp+0x140+0x30 (0x170)
mov rdi,rdx //moves rsp+0x140 to rdi
mov rbx,rcx //pointer to TP_POOL moved to rbx
lea rax,[rdi+0x10] //loads the address of rsp+0x140+0x10 to rax
lea rcx [rbx+0x60] //loads the address of the head of the list to rcx!!
mov rdx, [rcx+0x08]  //moves the address of the current tail to rdx
mov [rax],rcx //Saves the pointer to head of the list to rsp+0x140+0x10
              //Because the new worker is inserted to the tail of the list (fli
mov [rax+0x8],rdx // stores the address of previous tail into current structure
mov [rdx], rax            //oldTail->Flink = node
mov [rcx+0x08], rax       //head->Blink = node
```

Which means that by fetching TP_POOL of a worker factory we get access to the linked list that contains all structures that are always 0x38 bytes before our targeted TP_CALLBACK_INSTANCE object, we can easily fetch TP_CALLBACK_INSTANCE pointer by adding 0x28 the address stored in the list. This is a much better approach, no new handles opened, no loud context reading APIs and no thread enumeration.

So to recap the to-do list:

1. Find all TP_CALLBACK_INSTANCE objects in the pool we are going to post work to.
2. Inject the DLL path string into the beginning of TP_CALLBACK_INSTANCE
3. Set a callback function to LoadLibraryA
4. Push the callback into queue (just like normal poolparty)

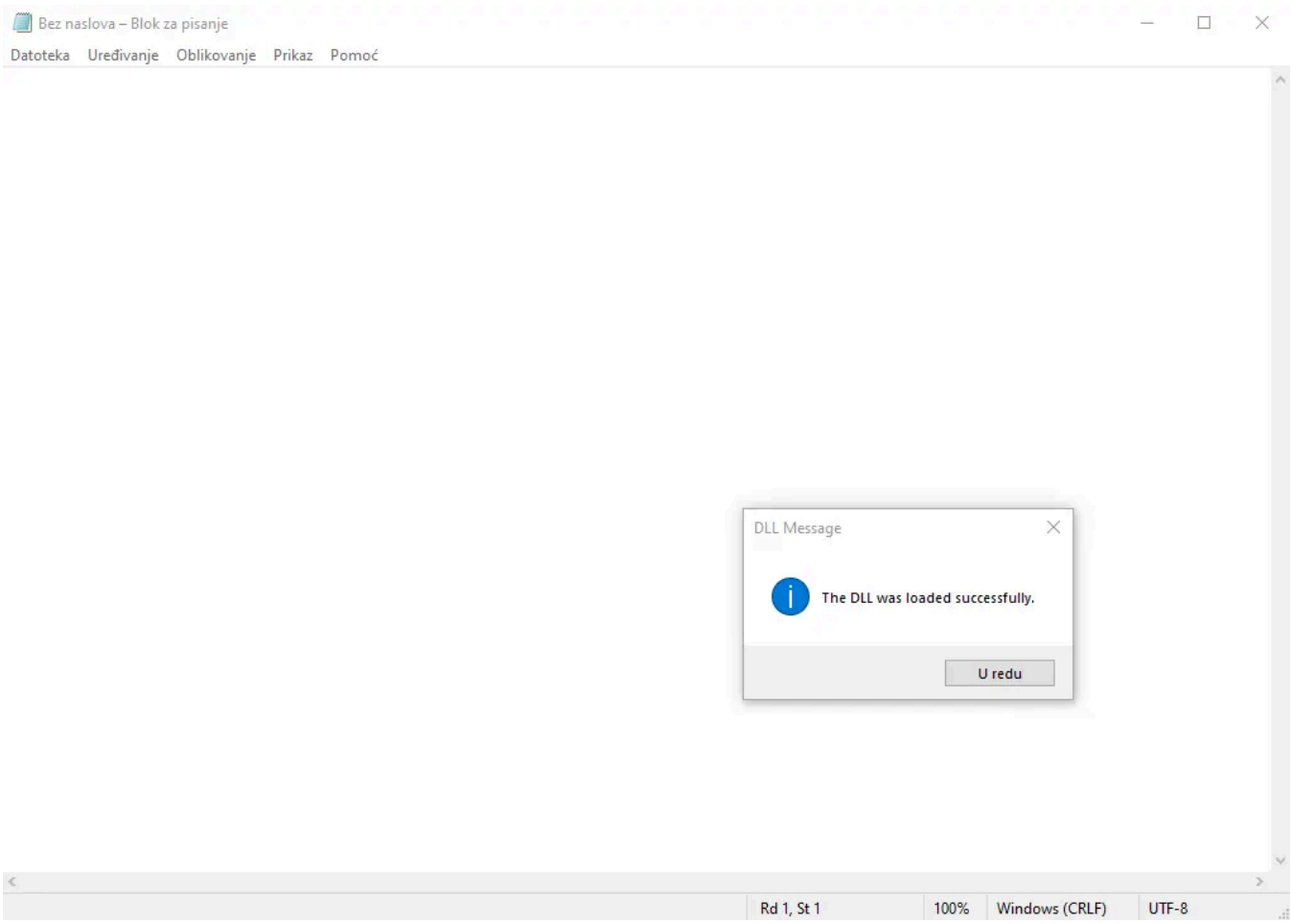
And it works!

```
PS D:\cpp\DllParty\poc> ps notepad

Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    243      13     3176      15224       0,06    5192   6 notepad


PS D:\cpp\DllParty\poc> .\DLLparty.exe 5192
[+] The requested process id:5192
[+] System information fetched!
[+] Worker Factory Object Type index found: 30
[+] Found the worker factory!
[+] Duplicated the worker factory handle!
[+] NtQueryInformationWorkerFactory resolved!
[+] Accuired TP_POOL object
[+] Worker count: 2[+] Fetched the IOCP port from workerFactory object
[!] All worker threads are inside the current pool, allocated new memory!
[+] Injection finished!
PS D:\cpp\DllParty\poc>
```

DLLparty execution



Dll injection into notepad

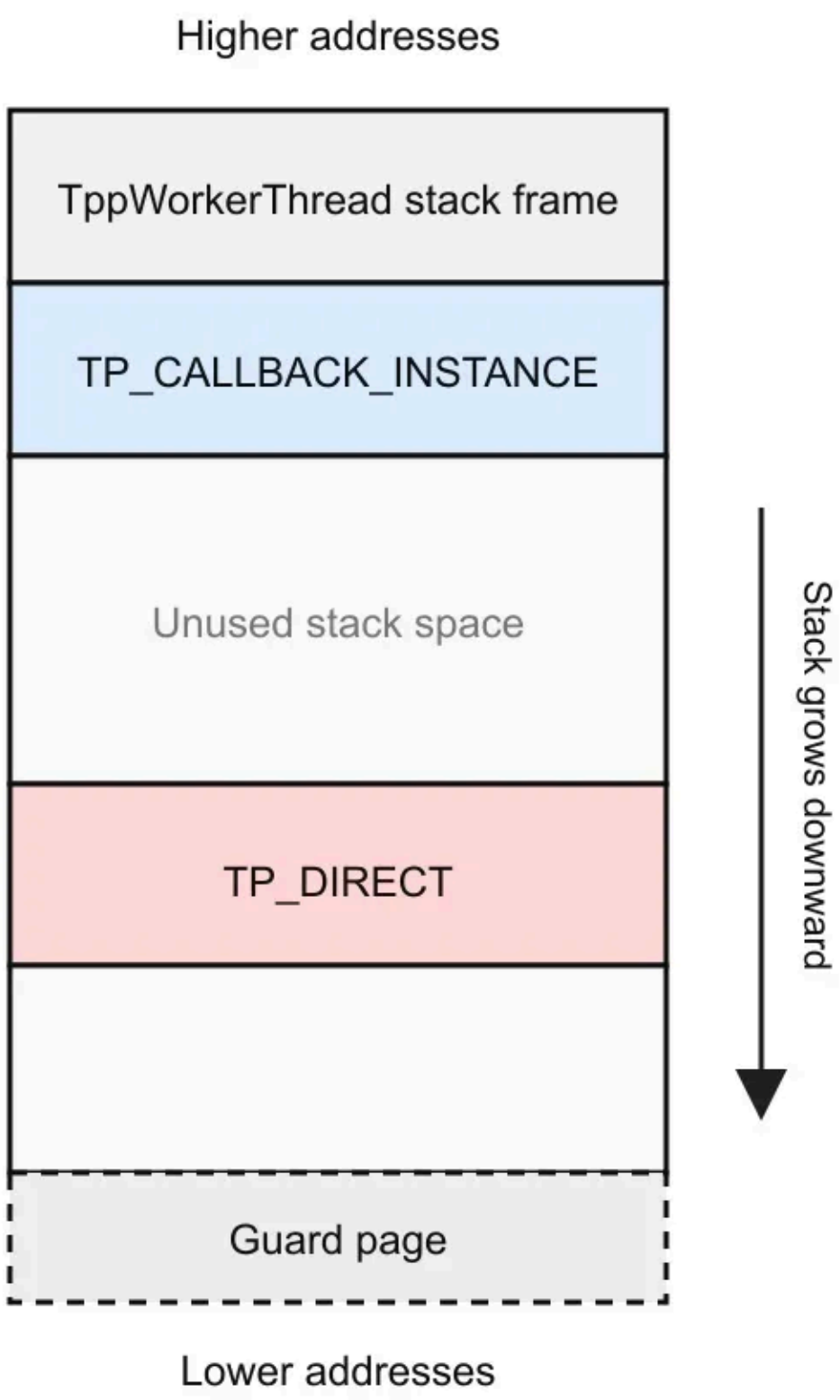
We get a callstack that looks like this:

```
13  ntdll.dll!LdrpPrepareModuleForExecution+0xbf
14  ntdll.dll!LdrpLoadDllInternal+0x19a
15  ntdll.dll!LdrpLoadDll+0xa8
16  ntdll.dll!LdrLoadDll+0xe4
17  KernelBase.dll!LoadLibraryExW+0x162
18  KernelBase.dll!LoadLibraryExA+0x31
19  KernelBase.dll!LoadLibraryA+0x3f
20  ntdll.dll!TppWorkerThread+0x456
21  kernel32.dll!BaseThreadInitThunk+0x14
22  ntdll.dll!RtlUserThreadStart+0x21
```

DLLParty callstack

Avoiding VirtualAllocEx

In the standard PoolParty technique VirtualAllocEx is called 2 times. First it's called to allocate space for the shellcode and second time it's called to allocate space for a variant specific TP structure that is pushed onto the IOCP queue. Let's take a loo at the TP_DIRECT PoolParty variant. TP_DIRECT object is created and pushed onto the IOCP queue. TP_DIRECT object is only 72 bytes big, not a lot of space is needed to store that. Since we already have pointers to READ/WRITE stack memory (TP_CALLBACK_INSTANCE pointers) we can choose one of those, subtract some offset(experimentally it seems 0x500 works fine) so that the object doesn't get overwritten by function calls that happen before the callback is invoked. We already don't have to call VirtualAllocEx for the shellcode allocation, because we aren't using any. And we can store the TP structure on the not yet used stack to avoid having to use VirtualAllocEx the second time.



Stack draft

Conclusion

This technique isn't actually just a DLL injection, it allows you to coerce a worker thread to call any function that matches chosen callback signature. If you take a quick look back to beginning of this blog you will see different signatures callback functions need to have depending on the type of callback we want to execute. Now we know that you can control PTP_CALLBACK_INSTANCE, the first parameter.

Second parameter is controllable by default. There are callback signature that have other parameters after these two, if we can also control all of those we can make worker threads execute even more functions, but this is a topic for some further research perhaps.

Exact offsets that I mentioned here can change from build to build, this was done on the latest Windows 10 build, but the general technique should remain the same for a longer time because it would require changing how TppWorkerThread works or changing TP_INSTANCE_CALLBACK structure.

The callstack where TppWorkerThread calls LoadLibraryA is still kinda suspicious. TppWorkerThread is a hardcoded compiled function, why is it calling LoadLibraryA? If you look at the source code there is a set number of functions it calls + the callback functions that are always user defined. The point is it is not suspicious enough, it's one legitimate kernel32 function calling another, no unbacked memory, no heavily monitored API calls, no triggering kernel callbacks. It might look super suspicious to us after this research but to an EDR or even an analyst that didn't go in depth on thread pool internals it likely won't. It certainly looks much cleaner than unbacked shellcode.

The POC that I provided is just a demonstration of the technique, if you want to use it in actual evasion scenarios make sure to modify the code with OPSEC in mind.

If anyone actually made it this far, thanks for reading and I hope you found this interesting! :D

PS: This is my first research blog, any and all criticism and suggestions are welcome!

By epersic

Cybersecurity + Process Injection + Malware Development +



Follow

Written by epersic